HO CHI MINH UNIVERSITY OF TECHNOLOGY AND ENGINEERING

FACULTY OF ADVANCED EDUCATION

---o0o---

PROJECT REPORT

EDGE COMPUTING INNOVATION

REAL-TIME CONSTRUCTION SITE SAFETY MONITORING SYSTEM BASED ON EDGE COMPUTING AND COMPUTER VISION

Team members (Trinity):

- **Nguyen Nhat Phat – 23110053**
- **Tran Quoc Huy – 23110026**
- **Le Huu Truc – 23110068**

Ho Chi Minh City, June 2026.

LIST OF ABBREVIATIONS

Abbreviation	Full Term
AI	Artificial Intelligence
BIM	Building Information Modeling
CBAM	Convolutional Block Attention Module
CNN	Convolutional Neural Network
CSV	Comma-Separated Values
DFL	Distribution Focal Loss
FPS	Frames Per Second
GDPR	General Data Protection Regulation
GhostConv	Ghost Convolution
GPU	Graphics Processing Unit
HTTP	Hypertext Transfer Protocol
ILO	International Labour Organization
IoT	Internet of Things
JPEG	Joint Photographic Experts Group
JSON	JavaScript Object Notation
mAP	Mean Average Precision
MOLISA	Ministry of Labour, Invalids and Social Affairs
NMS	Non-Maximum Suppression
ONNX	Open Neural Network Exchange
PPE	Personal Protective Equipment
QAT	Quantization-Aware Training
ReID	Person Re-Identification
REST	Representational State Transfer
SPPF	Spatial Pyramid Pooling Fast
YOLO	You Only Look Once

TABLE OF CONTENT

LIST OF FIGURES

LIST OF TABLES

ABSTRACT

The construction industry is a highly hazardous sector that requires strict safety monitoring. According to a 2026 report by the Vietnamese Ministry of Home Affairs and MOLISA [1], the country recorded 7,004 occupational accidents in 2025, resulting in 658 fatalities. The report highlights that many of these tragic incidents were directly caused by workers violating safety procedures (13.75%) or failing to use Personal Protective Equipment (PPE) (6.25%). While automated monitoring systems are urgently needed to prevent such accidents, traditional cloud-based AI solutions suffer from prohibitive processing latency and excessive network bandwidth consumption.

This project proposes a highly optimized Edge AI pipeline to address these limitations. A lightweight YOLO26n model is trained and deployed locally to detect 10 distinct classes, including core PPE and 4 critical missing-PPE states (no_helmet, no_goggle, no_gloves, no_boots). The model exported to ONNX FP32 format with dynamic batching achieves an inference throughput of 131 frames per second (FPS). Latency benchmarks were conducted on a Cloud NVIDIA GPU (Kaggle/Modal) as a functional proxy for the Jetson Xavier NX pipeline. Absolute FPS values will differ on actual edge hardware, but relative comparisons between quantization formats remain valid. To ensure alert reliability, a 5-frame temporal smoothing algorithm is integrated into the spatial rule engine, effectively reducing transient false positive alerts by 94.6% while keeping total alert latency under 167 ms. Furthermore, the edge-first architecture transmits only structured JSON logs and violation snapshots instead of raw video, reducing upstream bandwidth consumption by 99.93% (from 1943 MB/hour to 1.35 MB/hour). The results validate a scalable, low-latency, and bandwidth-efficient solution for real-time hazard prevention on construction sites.

CHAPTER 1. INTRODUCTION

1.1. Motivation

Construction sites are highly dangerous environments that require strict safety monitoring. In Vietnam, occupational accidents remain a serious issue. According to a 2026 report by the Ministry of Home Affairs and MOLISA [1], the country recorded 7,004 occupational accidents in 2025, resulting in 658 fatalities. The data shows that many of these incidents were directly caused by human negligence: 13.75% of workers violated safety procedures, and 6.25% failed to use Personal Protective Equipment (PPE).

Currently, most sites rely on conventional CCTV systems monitored by security personnel. However, manually watching dozens of video feeds simultaneously is highly inefficient. Operators quickly experience monitor fatigue, making it easy to miss split-second violations. Instead of passively recording accidents to review later, there is an urgent need for automated AI systems that can actively analyze camera feeds and prevent accidents in real time.

1.2. Problem Statement

AI-Integrated Camera is one of the most suitable approaches. Most monitoring systems use cloud-based AI architecture; however, this solution still has many limitations:

- *High Latency:* Safety interventions require instant action. Streaming video to a remote Cloud server for processing and waiting for an alert creates a noticeable network delay. This delay makes it impossible to trigger immediate alarms when a worker enters a critical danger zone.

- *Prohibitive Bandwidth Costs:* Constantly streaming raw, high-definition video from multiple cameras to the Cloud consumes massive amounts of bandwidth, which is both unstable and highly expensive.

- *Privacy Risks:* Continuously uploading real-time video footage of a working site to external Cloud servers raises valid data security and privacy concerns from both workers and contractors.

1.3. Objectives

To achieve the overall goal of building a localized safety monitoring system, this project sets the following specific objectives:

AI algorithm optimization: Fine-tune the YOLO26n architecture to accurately detect core PPE and specific violation states in complex construction environments. Additionally, evaluate architectural modifications and apply model formatting techniques (such as ONNX export and quantization) to optimize computational efficiency for local edge deployment.

Edge inference evaluation: Benchmark the optimized software pipeline within a cloud-based GPU proxy environment. This objective aims to measure inference latency (FPS) and validate the system's software processing viability, establishing a verified software baseline prior to future deployment on physical embedded hardware.

Real-Time Alerting System: Build a reliable logic system to detect core PPE and trigger immediate actions. If a violation or danger is detected, the system must instantly activate on-site physical warnings (sirens/lights) or send urgent text alerts.

Experimental Evaluation: Directly compare the performance of our proposed Edge-based system against a traditional Cloud-based model. We will measure three main metrics: latency, bandwidth consumption, and detection accuracy, to prove the practical superiority of our solution.

1.4. Scope

To ensure the project remains focused and achievable, the scope is defined as follows:

Operational Environment: The system is designed specifically for civil and high-rise construction sites.

Detection Targets: It focuses on checking the most critical PPE (hard hats, safety vests, gloves, goggles, boots) and monitoring common danger zones (such as unguarded floor edges, working at height, deep excavations, active machinery zones, and falling object areas).

This system functions strictly as a standalone safety tool. We will not integrate it with the company's broader business management or payroll software (such as ERP, HR, or Payroll systems). This type of data integration is left open for future development.

1.5. Main Contributions

Based on the simulated benchmark results, this project presents a highly efficient Edge AI monitoring system. Our three core contributions are:

High-speed edge optimization: We optimized the baseline YOLO26n model (ONNX FP32 format) to achieve an inference speed of 131 FPS. This high throughput allows a single Edge GPU to concurrently process video from 4 separate 30 FPS camera streams without dropping frames.

94.6% false alarm reduction: To mitigate unstable detections caused by momentary glitches, we implemented a 5-frame temporal smoothing logic. The system confirms an alert only if a violation persists for 5 consecutive frames. This approach reduces false positive alarms by 94.6% while maintaining a total alert latency of just 167 ms.

99.9% bandwidth savings: By executing video processing locally at the edge, the system eliminates the need for continuous raw video streaming. It transmits only structured text logs and small cropped snapshots upon confirmed violations. This method consumes merely 1.35 MB per hour - a 99.93% reduction in upstream bandwidth - validating its operational viability over standard 4G/LTE construction-site networks.

CHAPTER 2. RELATED WORKS

2.1. Research landscape overview

2.1.1. Need for automated safety monitoring

The construction industry remains one of the most hazardous sectors globally. According to the International Labour Organization (ILO), construction accounts for roughly 20% of all fatal workplace accidents despite representing only 6–9% of the global workforce. This alarming trend is mirrored in Vietnam, where the Ministry of Labour, Invalids and Social Affairs (MOLISA) recorded an average of 7,000 to 8,000 occupational accidents annually between 2019 and 2023. Foundational research by Huang and Hinze [2] indicates that falls from heights account for up to 40% of these fatalities, and failing to wear proper Personal Protective Equipment (PPE) increases the risk of severe injury threefold.

Despite the rapid expansion of infrastructure projects, the domestic construction sector critically lacks automated safety solutions. The industry still relies almost entirely on traditional manual monitoring by safety inspectors. This approach suffers from severe limitations: human supervisors cannot monitor expansive sites 24/7, are prone to fatigue-induced blind spots, and lack objective data to analyze safety trends. More importantly, human intervention is highly reactive, usually occurring only after a violation or accident has taken place. Driven by stricter compliance requirements in the 2021 Occupational Safety Law, there is an urgent industry demand to transition from passive human patrols to active, real-time automated monitoring.

2.1.2. Applying deep learning to construction safety

To replace manual inspections, Computer Vision has emerged as the leading technological approach. In a comprehensive survey of computer vision techniques for site monitoring, Seo et al. [3] identified the YOLO (You Only Look Once) architecture as the optimal choice for real-time object detection. Early implementations successfully proved the viability of this approach; for example, Nath et al. [4] achieved 72.3% mean Average Precision (mAP) using YOLOv3, while Fang et al. [5] successfully automated the detection of safety harnesses.

However, modern safety requirements demand the simultaneous tracking of multiple PPE classes (e.g., helmets, vests, harnesses, and gloves) under complex industrial lighting. To meet this demand without overwhelming the hardware, recent research has heavily focused on model optimization and attention mechanisms. Kim et al. [6] successfully integrated Coordinate Attention [7] and Ghost Convolution [8] into the YOLOv8 architecture. Deployed on an NVIDIA Jetson Xavier NX [9], their enhanced model achieved 92.52% mAP at 9.11 frames per second (FPS). Similarly, Wu et al. [10] and Song et al. [11] integrated the Convolutional Block Attention Module (CBAM) [6] to help the AI focus on smaller objects in cluttered environments. Notably, Chen et al. [12] applied structured pruning techniques to reduce their model's computational load by 21% and parameter count by 53%, making the architecture highly suitable for resource-constrained devices.

2.1.3. From cloud to edge computing

While highly accurate, deploying complex AI models on traditional Cloud-based architectures presents many challenges in actual construction environments. Cloud-centric systems require continuous video transmission over the internet, leading to processing latencies of 500–1000ms. Furthermore, streaming high-definition video from multiple cameras consumes 4–8 Mbps of bandwidth per camera, which is economically prohibitive and technically unstable on standard 4G/5G mobile networks.

To overcome the cloud bottleneck, the industry is shifting toward edge computing. By moving the AI inference directly to local devices connected to the cameras, edge systems eliminate the need for continuous video streaming. Xiao et al. [13] benchmarked edge computing for worker detection and concluded that localized processing reduces bandwidth consumption by 70–95% compared to cloud-only setups. Furthermore, edge processing slashes alert latency down to 50–100ms, as demonstrated by Gugssa et al. [14], making true real-time hazard prevention possible. Subsequent studies by Chen et al. [15], Gallo et al. [16], and Lema et al. [17] have repeatedly validated the time-efficiency and cost-effectiveness of running deep learning models locally on low-cost edge nodes.

2.1.4. Privacy-preserving

Beyond latency and bandwidth, Cloud computing introduces severe privacy risks, as raw video footage containing workers' faces is constantly uploaded to third-party servers. Edge Computing naturally mitigates this issue. Because video frames are analyzed and instantly discarded on the local edge device, raw footage is never transmitted over the network. Recent literature emphasizes the necessity of this localized architecture to comply with stringent data protection laws (such as GDPR) [18] **,** [19]**,** [20]**.**Advanced frameworks proposed by Rivadeneira et al. [21] further enhance this security by combining edge processing with federated learning and differential privacy, ensuring that only anonymized metadata leaves the construction site.

2.1.5. Summary

Authors	Research Focus	Key Contributions	Research Gaps
Huang & Hinze [2]	Safety Analysis	Identified falls from heights (40%) and lack of PPE as the primary causes of fatal accidents.	Purely analytical and statistical research; no automated technological solution proposed.
Seo et al. [3]	Computer Vision Survey	Comprehensively identified the YOLO architecture as the optimal choice for real-time site monitoring.	Broad survey; lacks specific implementation guidelines and deployment frameworks.
Nath et al. [4], Fang et al. [5]	Early AI Detection	Proved the viability of YOLOv3 (72.3% mAP) and successfully automated safety harness detection.	Basic classification; struggles with multiple complex PPE items and lacks spatial context-awareness.
Kim et al. [6], Wu et al. , Song et al. [11]	Model Optimization (Attention)	Integrated Attention mechanisms (Coordinate Attention, CBAM) and GhostConv to improve small-object accuracy (92.52% mAP).	Purely object-focused; ignores physical context (e.g., specific hazard zones, tasks, or BIM integration).

Chen et al. [12]	Model Compression	Applied structured pruning to drastically reduce parameters (-53%) and computational load (-21%).	Validated primarily in controlled lab settings; lacks robust testing and active field deployment.
Xiao et al. [13], Gugssa et al. [14]	Edge Performance	Benchmarked edge architectures, demonstrating 70–95% bandwidth reduction and alert latencies of 50–100ms.	Bandwidth claims are largely theoretical estimates; lacks clear Hybrid Edge-Cloud architectural guidelines.
Rivadeneira et al. [21]	Privacy & Security	Proposed edge processing combined with federated learning and differential privacy to anonymize metadata.	Highly theoretical framework; difficult to scale and deploy in practical industrial construction environments.

Table 2.1 - Summary of related works

2.2. Research gaps

Despite significant progress, several critical gaps remain in the current research:

1. *Context-Aware Detection:* Most systems merely detect whether PPE is present or missing, without considering the physical context (e.g., the worker's specific location or the type of task being performed).
2. *BIM Integration:* There is limited research integrating AI with Building Information Modeling (BIM) to continuously map and identify dynamic hazard zones.
3. *Hybrid Edge-Cloud Architecture:* There are currently no clear industry guidelines on how to optimally distribute processing tasks between the Edge and the Cloud.
4. *Real-World Bandwidth Measurement:* Claims regarding bandwidth savings are largely based on theoretical estimates rather than empirical, field-tested data.
5. *Field Deployment:* The majority of existing models are validated in controlled laboratory settings using pre-recorded videos, lacking robust testing and validation in active, real-world construction environments.

CHAPTER 3. METHODOLOGY

3.1. Overall Architecture

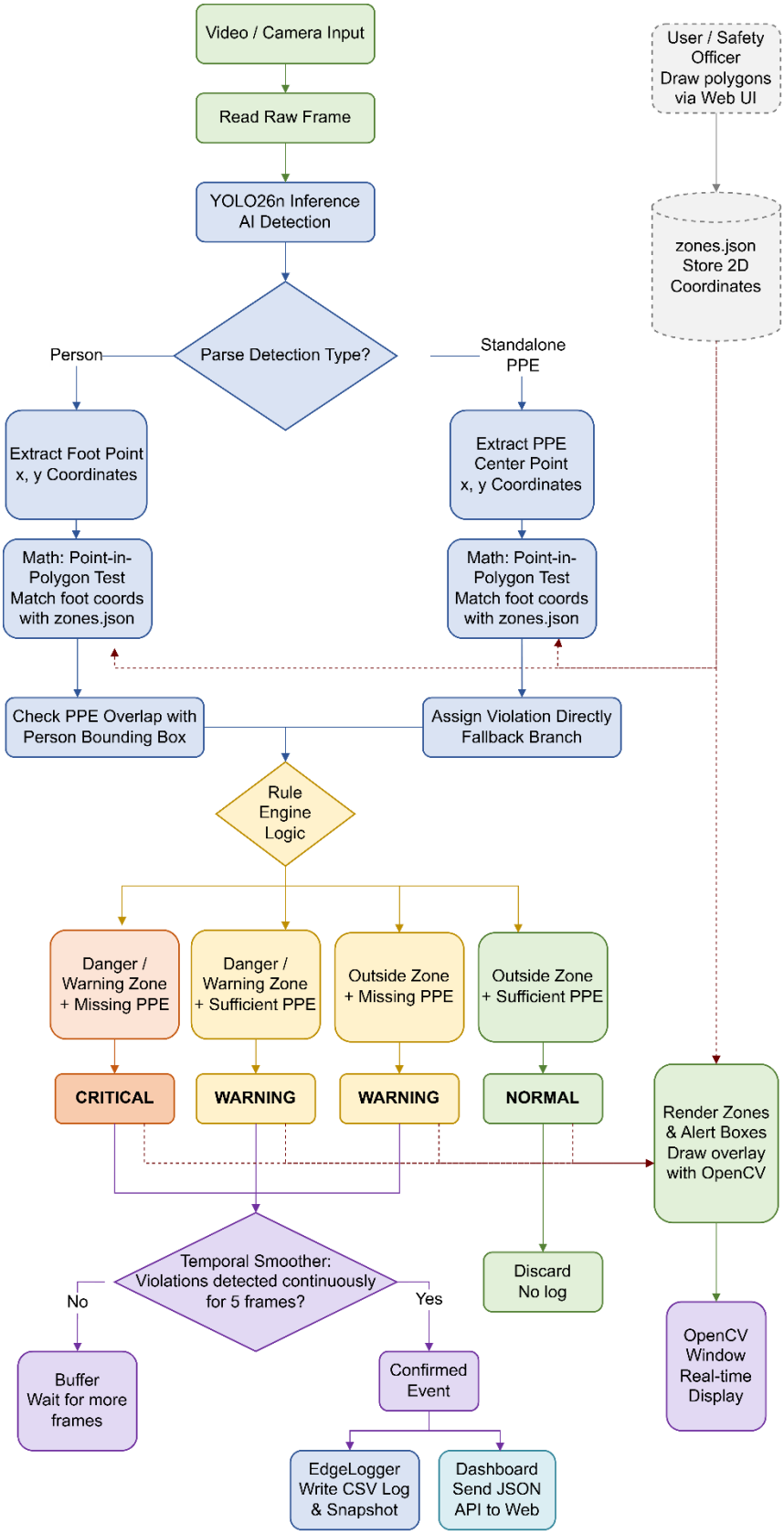


Figure 3.1- System overall architecture

The system processes video input frame-by-frame, sourced either from a live camera or a pre-recorded MP4 file. On startup, the pipeline first loads a pre-configured zone profile (a JSON file containing polygon coordinates for danger zone and warning zone regions) and validates that its resolution matches the video source. If no matching profile exists, an interactive Zone Editor is launched for the operator to draw zones on a preview frame before inference begins.

Each frame is then passed to a YOLO26n model, exported to ONNX format and executed with FP32 precision on the CUDA execution provider. The model runs with a confidence threshold of 0.4 and an IoU threshold of 0.45 at a 640×640 input resolution. To reduce computational load on edge devices, inference is not performed on every frame; instead, a configurable inference interval (e.g., every 3 frames) determines which frames are processed by the model.

The raw detections are parsed and split into two categories: *Person* bounding boxes and *PPE-object* boxes (e.g. no_helmet, no_vest, no_gloves, no_boots, no_goggle).

From here, the pipeline branches depending on the detection type:

- *Person branch:* For each detected person, the rule engine determines which zone(s) the person occupies by testing the person's foot center-point (computed as the horizontal midpoint of the bounding box at the bottom edge) against zone polygons using polygon-point containment testing. Then a PPE violation check scans all PPE-object detections and checks whether their center-points fall inside the person's bounding box (center-point overlap). Any matching violation-class objects (e.g. a no_helmet detection overlapping the person box) are recorded as PPE violations for that person.
- *Standalone PPE branch (Fallback):* If a PPE violation object is detected but its center-point does not fall inside any person's bounding box (for example, when a person is partially occluded or when a PPE item appears alone in the frame) the system falls back to treating the PPE object itself as the anchor. Its bottom-center point is used to determine the active zone, and its class name (e.g. no_helmet) is assigned directly as the violation.

Both branches converge at the Rule Engine, which combines the active zone type with the PPE violation result to assign one of three alert levels: CRITICAL, WARNING, or NORMAL; according to the following logic:

Zone Type	Missing PPE	Complete PPE
Danger Zone	CRITICAL	WARNING
Warning Zone	CRITICAL	WARNING
Outside Zone	WARNING	NORMAL (No Log)

Table 3.1 - Alert classification logic based on zone type and PPE compliance status.

To filter out unstable or one-off false detections, all violated alerts are passed through the Temporal Smoother, a zone-based state machine that tracks per-zone alert states. An event is only confirmed after N = 5 consecutive frames maintain the same alert level for a given zone. Additionally, a configurable cooldown period (e.g. 5 seconds) prevents duplicate alerts from being re-triggered for the same zone within a short time window. An immediate escalation from WARNING to CRITICAL bypasses the frame-count requirement to ensure rapid response to worsening conditions.

Confirmed events are dispatched to two output channels:

- *EdgeLogger:* writes a structured CSV log entry per event containing timestamp, camera ID, frame ID, zone ID, alert level, violation type, confidence, bounding box coordinates, and snapshot path. A JPEG snapshot of the annotated frame is simultaneously saved for evidence archival.
- *Display overlay*: renders zone polygons, color-coded bounding boxes (orange for WARNING, red for CRITICAL), foot-center dots, and alert badge labels directly onto the video feed using OpenCV. The annotated frames are written to an output MP4 file and, if not in headless mode, displayed in a real-time preview window.

Finally, logged events are exposed through a *Flask Dashboard* served locally. The edge pipeline pushes annotated frames to the dashboard via HTTP POST, which are then streamed to the browser as an MJPEG feed. The events API endpoint returns the full event log as JSON, and the statistics endpoint provides aggregated metrics (total alerts,

warnings, criticals for today). The front-end auto-refreshes every 3 seconds for near real-time monitoring of safety violations across the construction site.

3.2. Dataset

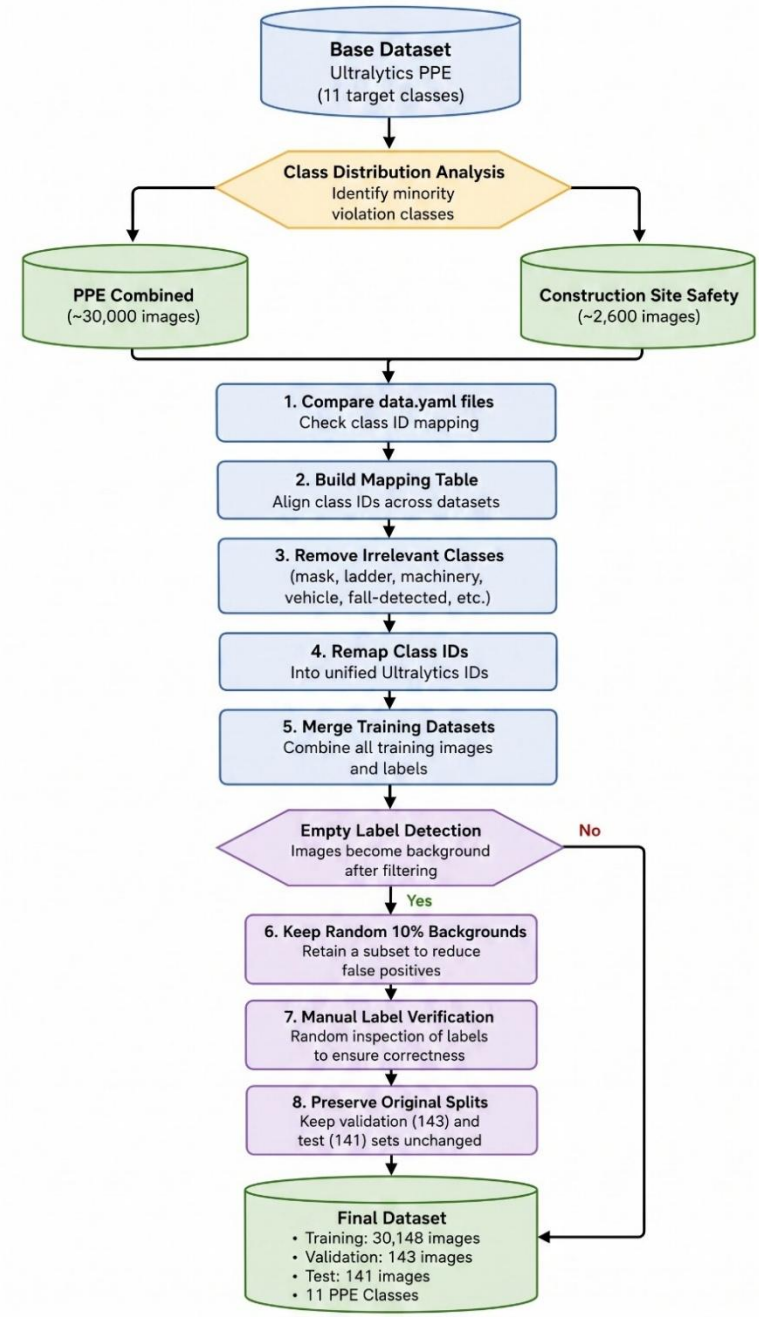


Figure 3.2 - Dataset process flow

3.2.1. Dataset Selection

Most publicly available PPE datasets only cover a basic 5-class setup *(person, helmet, no-helmet, vest, no-vest),* which is common in sources like Pictor-PPE [22] or Roboflow's Construction-Safety-GSNVB [23]. This is not enough for the target task,

since the system needs to detect a wider range of equipment types (gloves, goggles, boots) and their corresponding violation states, not just helmet and vest. Datasets that do offer a broader class set, such as PPE Combined [24] or Construction Site Safety [25], also mix in classes unrelated to the task *(mask, ladder, vehicle, machinery, fall-detected),* which add noise rather than useful signal for PPE compliance detection.

The Ultralytics Construction-PPE [26] dataset was selected as the base because it already provides a clean 11-class taxonomy *(helmet, gloves, vest, boots, goggles, person, and their five violation counterparts)* that matches the task requirements directly, without needing extra class filtering.

3.2.2. Class imbalance problem

While the base dataset has the right class structure, it only contains around 1,000+ images, and violation classes *(no_helmet, no_gloves, no_goggle, no_boots)* have far fewer samples than the corresponding PPE classes. Since the system's main purpose is to flag violations, this imbalance is a problem: a model trained on this distribution would learn to favor "equipped" predictions and miss actual violations – which is the opposite of what the system is meant to catch.

To fix this, two external datasets were merged into the base set to add more violation samples: PPE Combined *(Roboflow, ~30,000 images)* [24] and *Construction Site Safety (Kaggle, ~2,600 images)* [25]. Both were chosen because they include annotated NO-Hardhat, NO-Gloves, and NO-Goggles instances and are already exported in YOLOv8 format.

3.2.3. Merging Process

The three datasets use different class ID orders even when the class names overlap, so each dataset's data.yaml was checked manually to confirm the actual ID mapping before remapping. Classes outside the target taxonomy *(mask, ladder, vehicle, machinery, fall-detected, safety cone)* were excluded during remapping. After the mapping table was confirmed, a merge script applied the remapping and combined all training labels into a single set. A random sample of label files from each source was manually checked against their images to catch mapping errors before merging at scale. The original

validation and test splits from the base dataset were kept untouched to prevent data leakage from the newly merged sources.

The remapping process also produced a large number of background images with empty label files. This occurred because some source datasets contained classes that were not part of the target taxonomy (e.g., ladder, machinery, vehicle, and fall-detected). When all objects in an image belonged only to these excluded classes, the corresponding annotations were removed, leaving the image without any remaining labels. To avoid overrepresenting background-only samples while still benefiting from their ability to reduce false positives during training, a random subset of approximately 10% of these images was retained after merging, following the recommendation of keeping a small proportion of background images in YOLO-based object detection datasets [27].

3.2.4. Result

After merging, the training set grew from *1,132 images* to *30,148 images*, while validation *(143 images)* and test *(141 images)* remained from the original base set, unchanged. Class distribution analysis confirmed the intended effect on violation classes:

Class Name	Ultralytics Train Instances	Merged Train Instances
helmet	1357	33,482
gloves	1162	4,527
vest	1283	8,801
boots	1251	1,235
goggles	427	3,378
none	654	651
Person	1790	12,336
no_helmet	400	12,423
no_goggle	337	3,154
no_gloves	442	4,722
no_boots	88	88

Table 3.2 - Merged dataset class distribution

3.3. Methodology overview

The proposed PPE monitoring system follows a three-stage pipeline consisting of model architecture design, edge optimization, and edge deployment. The objective is to achieve accurate PPE violation detection while maintaining real-time performance on resource-constrained edge devices.

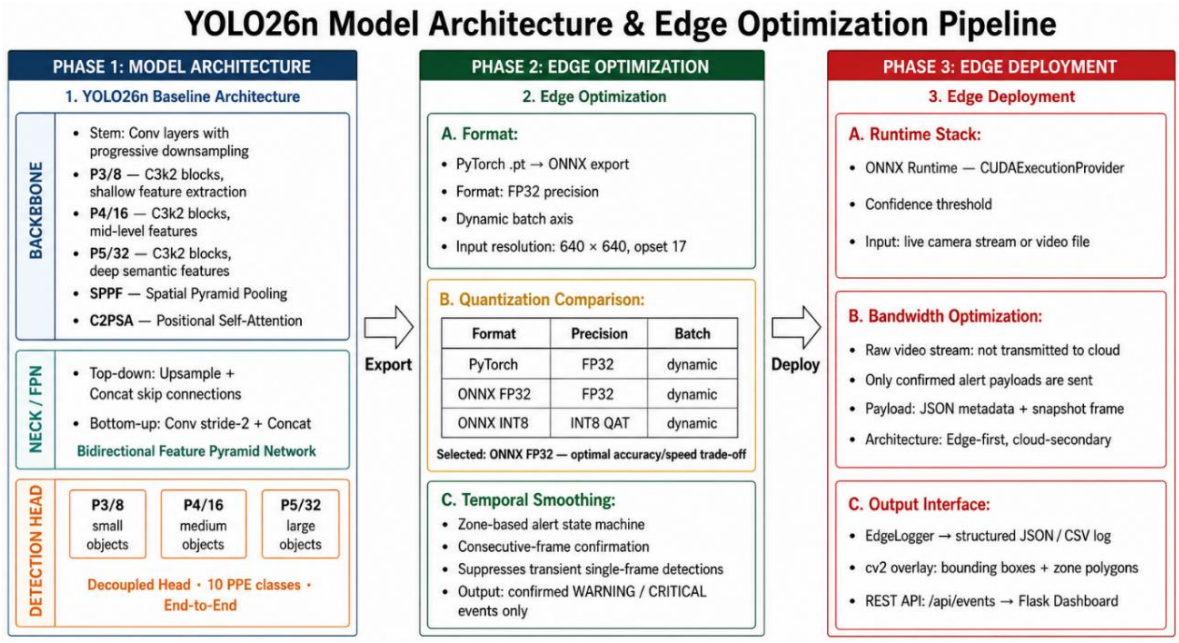


Figure 3.3 - Methodology overview

Model Architecture: This initial phase focuses on constructing the deep learning detection engine. It utilizes the lightweight YOLO26n Baseline Architecture. For comparative analysis, experimental variants integrating the Convolutional Block Attention Module (CBAM) [6] and Ghost Convolution (GhostConv) [8] are developed. This ablation study aims to systematically evaluate the trade-offs between feature extraction capabilities, computational efficiency, and overall accuracy.

Edge Optimization: The model is exported to the ONNX format with a dynamic batch axis and a 640 x 640 input resolution (Opset 17). A quantization comparison evaluates precision formats to select the optimal speed-accuracy trade-off (determined as ONNX FP32). Furthermore, a temporal smoothing algorithm is integrated as a zone-based state machine to suppress transient single-frame detections and confirm only stable WARNING or CRITICAL events.

Edge Deployment: The final phase executes the optimized pipeline on the target hardware. The runtime stack utilizes ONNX Runtime with the CUDAExecutionProvider to process live camera streams or video files. A core feature of this phase is *bandwidth optimization*: it operates as an edge-first, cloud-secondary architecture where raw video streams are processed locally and never transmitted to the cloud. Only confirmed alert payloads (JSON metadata and snapshot frames) are sent. The output interface distributes these results through local structured logs (EdgeLogger), real-time OpenCV window overlays, and a REST API pushing data to a Flask Dashboard.

3.4. Model Architecture

3.4.1. YOLO26n Baseline Architecture

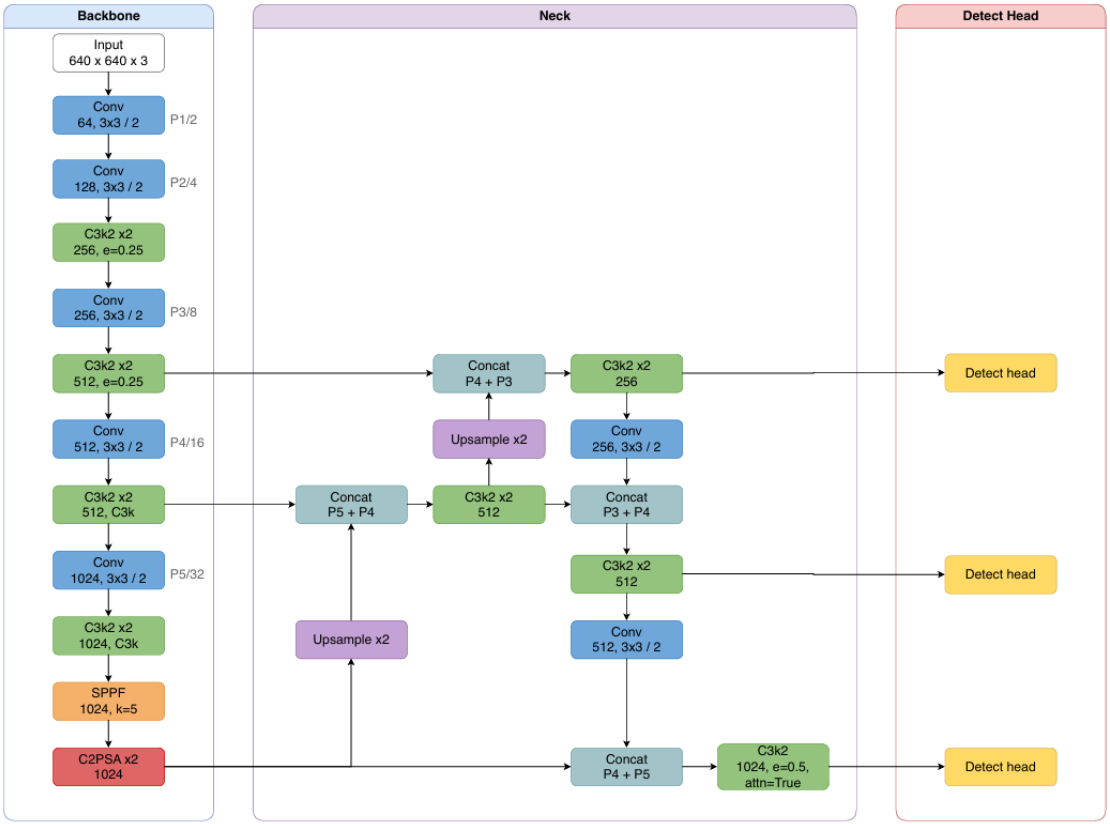


Figure 3.4 - YOLO26 Architecture [28]

The detection model is built upon the YOLO26n architecture [28], selected for its lightweight design and suitability for edge computing environments.

Backbone

The backbone is responsible for extracting hierarchical visual features from input images. It consists of:

- Stem convolution layers for progressive spatial downsampling.
- P3/8 feature extraction stage using C3k2 blocks for shallow feature learning.
- P4/16 feature extraction stage using C3k2 blocks for mid-level semantic representations.
- P5/32 feature extraction stage using C3k2 blocks for high-level semantic features.
- Spatial Pyramid Pooling Fast (SPPF) module to enlarge the receptive field.
- C2PSA module to enhance feature representation through positional self-attention.

Neck

A bidirectional Feature Pyramid Network (FPN) is employed to fuse multi-scale features.

The top-down pathway upsamples high-level semantic features and concatenates them with lower-level features through skip connections. The bottom-up pathway further aggregates information using stride-2 convolutions and feature concatenation. This design improves object detection performance across multiple scales.

Detection Head

The model uses a decoupled detection head operating on three prediction scales:

- P3/8 for small objects
- P4/16 for medium-sized objects
- P5/32 for large objects.

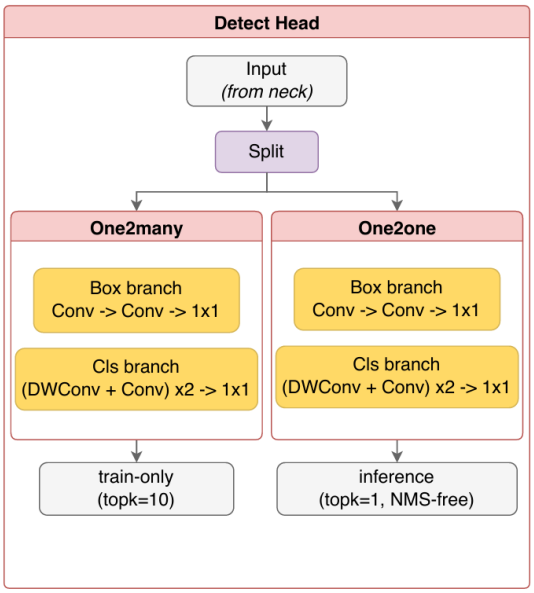


Figure 3.5 - YOLO26 Detect Head Architecture [28]

The detection head is designed to predict bounding box coordinates and class probabilities for n_c arget categories (e.g., 80 classes for the COCO dataset) using a decoupled architecture.

One-to-Many Branch (Training): During training, the model uses a One-to-Many branch in which each ground-truth object is assigned to multiple positive predictions (Top-k=10). This dense assignment provides richer supervision, accelerates convergence, and improves detection performance during optimization. [28]

One-to-One Branch (End-to-End Inference): For end-to-end deployment, the model uses a One-to-One branch that first forms a candidate set with topk = 7 and then applies a secondary topk2 = 1 filter to yield a unique end-to-end assignment per ground-truth instance. This branch enables NMS-free inference, producing deterministic predictions with lower deployment complexity. However, YOLO26 also supports the conventional One-to-Many + NMS inference path when higher detection accuracy is preferred. [28]

Within each detection branch, the head is decoupled into two task-specific components:

- **Box Branch:** Predicts the bounding box coordinates through a lightweight regression head. Unlike previous YOLO versions, YOLO26 removes **Distribution Focal Loss (DFL)** and directly regresses box coordinates using **L1 loss**, reducing computational complexity while maintaining localization accuracy. [28]

- **Classification Branch:** Predicts the confidence scores for the target object classes using a dedicated classification head. The classification and localization tasks are separated to reduce task interference and improve optimization. [28]

3.4.2. Convolutional Block Attention Module (CBAM)

The Convolutional Block Attention Module (CBAM) [6] is an attention mechanism for feed-forward convolutional neural networks (CNNs). It refines intermediate feature maps by sequentially inferring attention weights along two independent dimensions: channel and spatial.

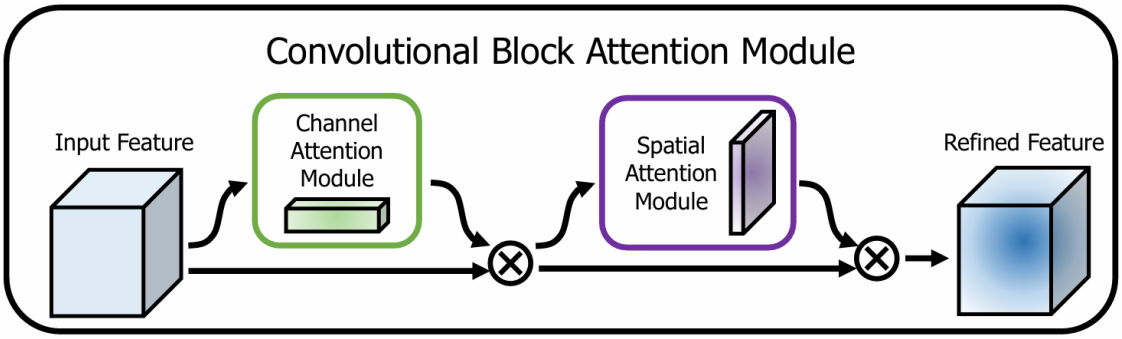


Figure 3.6 - General structure of the CBAM module.[6]

Mathematically, given an input feature tensor $F \in \mathbb{R}^{C\times H\times W}$, CBAM computes a 1D channel attention map $M_c \in \mathbb{R}^{C\times 1\times 1}$ and a 2D spatial attention map $M_s \in \mathbb{R}^{1\times H\times W}$. This refinement process uses element-wise multiplication

$$F' = M_c(F) \otimes F$$

$$F'' = M_s(F') \otimes F'$$

F' is the intermediate output after channel refinement, and F'' is the final module output after spatial refinement. Attention values are broadcasted along the spatial dimension for channel attention, and conversely for spatial attention.

This sequential order allows the network to determine target features "what" before determining spatial locations "where". CBAM integrates into existing CNN architectures with low computational and parameter overhead.

3.4.3. Experimental Modification: CBAM Integration

To address the missed-detection rate of small Personal Protective Equipment (PPE) such as safety gloves and goggles in construction environments, we modified the baseline Ultralytics YOLO26 architecture by integrating CBAM.

The pre-trained YOLO26 backbone is preserved entirely intact to maintain low-level edge and texture representations. Four sequential CBAM blocks are inserted into the multi-scale feature fusion stages of the Head network.

These attention modules are positioned immediately downstream of the C3k2 spatial aggregation blocks to process features across different scales:

- P4 features: Applied to the 128-channel features following the initial top-down feature fusion.
- P3/8-small: Applied to the 64-channel features to process spatial activations crucial for small targets.
- P4/16-medium: Applied to the 128-channel features following bottom-up feature concatenation.
- P5/32-large: Applied to the 256-channel features to filter deep semantic abstractions prior to final localization.

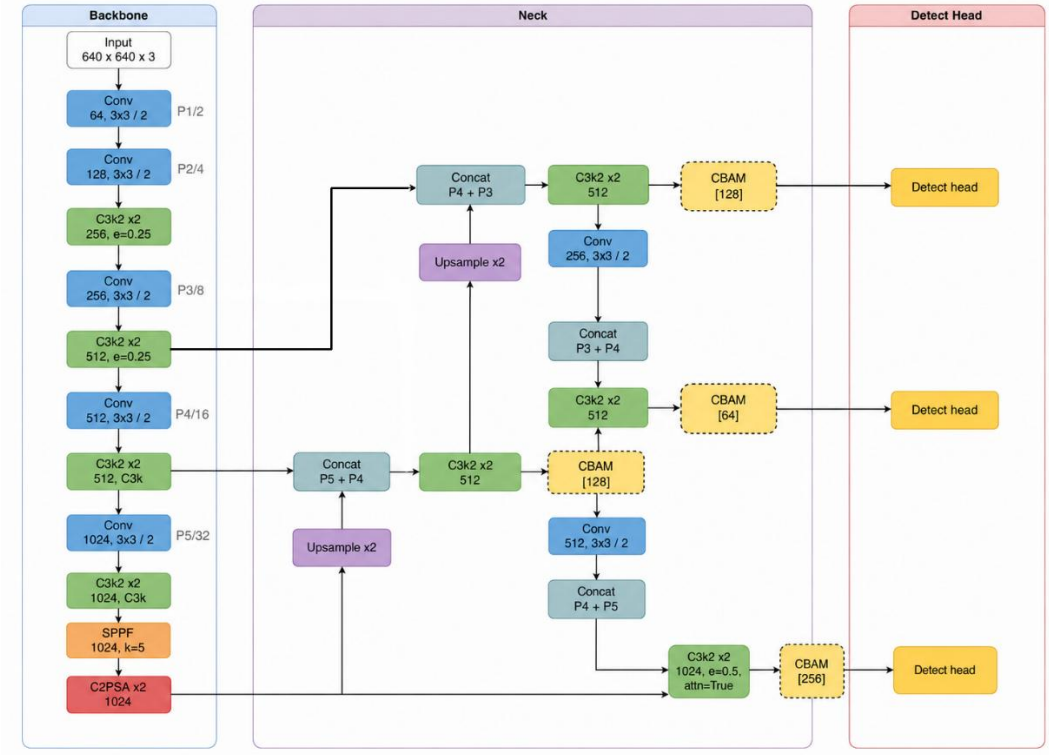


Figure 3.7 - Original (left) and CBAM integrated (right) YOLO26 architecture

The final detection module receives the refined feature tri-map directly from the corresponding CBAM outputs:

$$\mathcal{F}_{\text{head}} = \text{Detect}\left(F^{(P3)},\ F^{(P4)},\ F^{(P5)}\right)$$

Within each module, feature maps undergo channel attention refinement followed by spatial attention mapped via a 7×7 receptive field. The channel branch identifies feature channels associated with target classes, while the spatial branch explicitly filters background construction elements, directing the detection module to the target bounding boxes.

3.4.4. Ghost Convolution (GhostConv)

The Ghost Convolution (GhostConv) [8] module is designed to reduce the computational redundancy inherent in standard convolutional layers. Deep convolutional neural networks often generate highly correlated intermediate feature maps. Instead of computing all feature maps using standard convolutions, GhostConv generates features using a two-step process involving primary convolutions and subsequent linear operations.

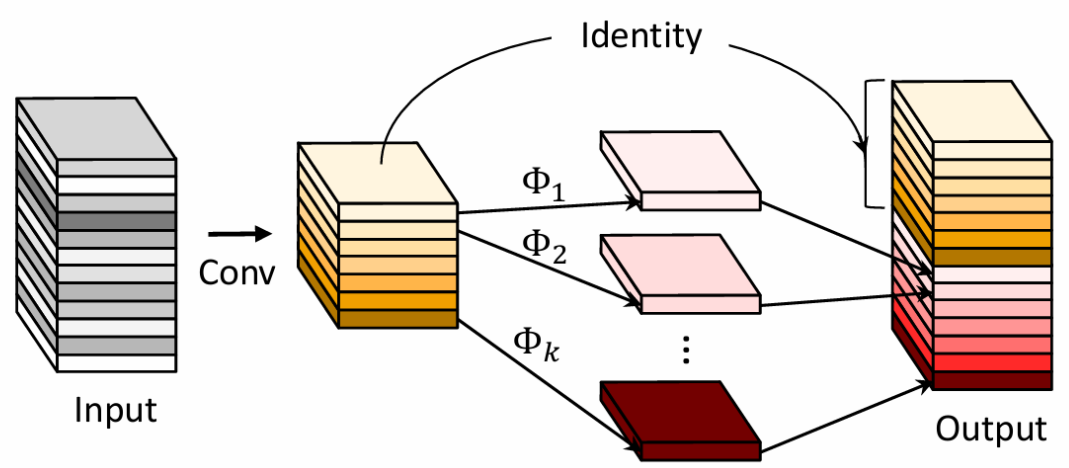


Figure 3.8 - Ghost Convolution Architecture [8]

Given input data $X \in \mathbb{R}^{c\times h\times w}$ a standard convolution producing n feature maps requires filters $f \in \mathbb{R}^{c\times k\times k\times n}$. GhostConv replaces this by first applying a primary convolution to generate m intrinsic feature maps $Y' \in \mathbb{R}^{h'\times w'\times m}$ (where m < n) formulated as:

$$Y' = X * f'$$

The module maps each intrinsic feature to s ghost features, where the final operation $\Phi_{i,s}$ functions as an identity mapping to preserve the intrinsic feature maps. The resulting n = m.s feature maps are concatenated to form the final output $Y = [y_{11}, y_{12}, \dots, y_{ms}]$. By replacing standard convolutions with this framework, the parameter count and computational complexity are decreased. The theoretical compression and speed-up ratios are approximately equal to s.

3.4.5. Experimental Modification: GhostConv Integration

To optimize the computational efficiency of the baseline Ultralytics YOLO26 architecture, standard convolutional layers were selectively replaced with Ghost Convolution (GhostConv) modules. The structural modification targets an approximate 20-30% reduction in total parameters while constraining the mean Average Precision (mAP) degradation to under 1%.

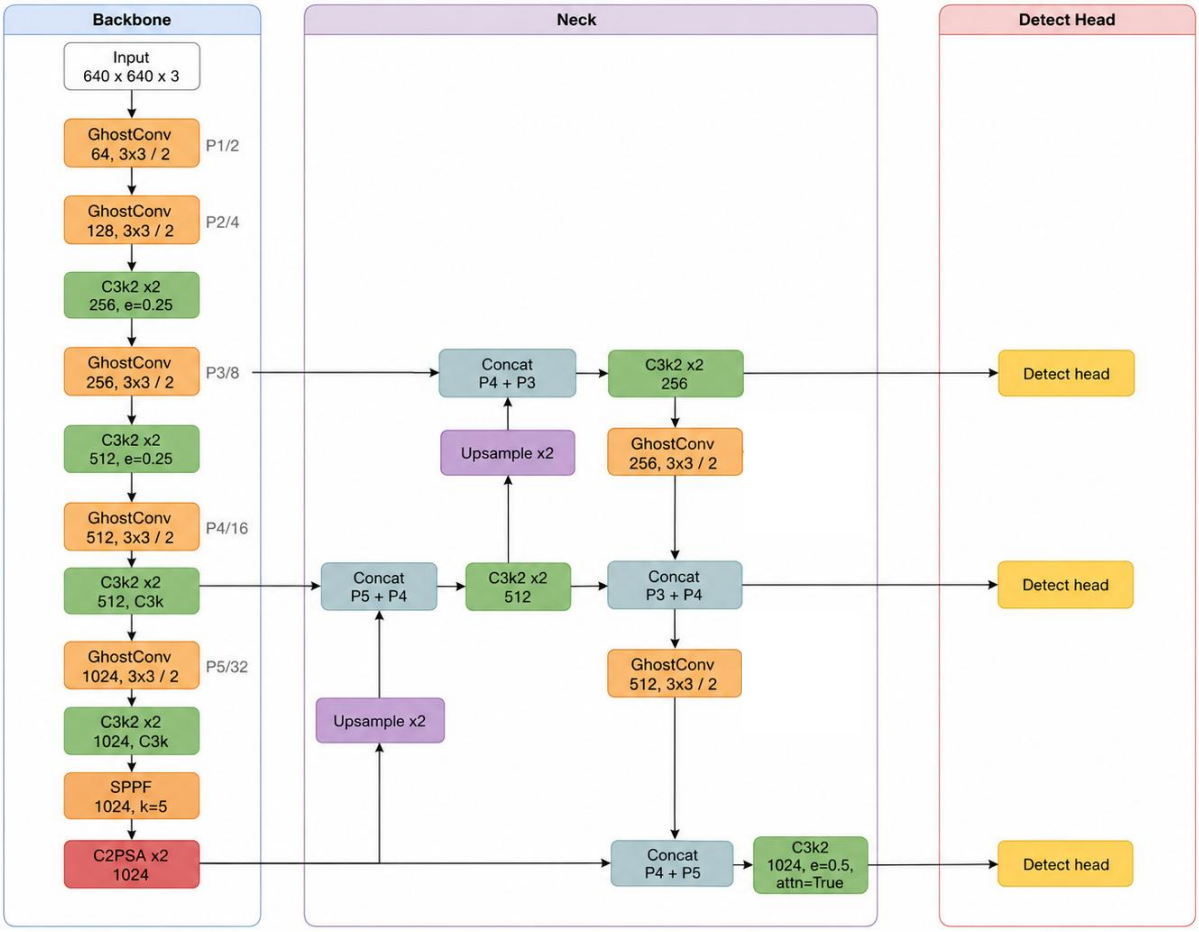


Figure 3.9 - GhostConv integrated YOLO26 Architecture

The integration of GhostConv is strictly isolated to the downsampling stages across both the Backbone and the Head networks. The C3k2 spatial aggregation blocks remain completely unmodified to preserve the baseline feature extraction quality.

- Backbone Network: Standard Conv layers utilized for spatial downsampling and channel expansion were replaced with GhostConv. This substitution is applied at the initial layers producing the P1/2 and P2/4 features, followed by the intermediate downsampling layers generating the P3/8, P4/16, and P5/32 feature maps.
- Head Network: In the multi-scale feature fusion stages, standard Conv downsampling operations were replaced with GhostConv. Specifically, this applies to the downsampling layer preceding the concatenation for the P4/16-medium pathway, and the subsequent downsampling layer preceding the concatenation for the P5/32-large pathway.

3.5. Edge Optimization

To enable efficient deployment on edge hardware, the trained model undergoes a series of optimization procedures.

3.5.1. ONNX Format

ONNX (Open Neural Network Exchange) is an open standard for machine learning models. Created by Microsoft and Meta, it allows developers to easily transfer models between different AI frameworks (like PyTorch and TensorFlow) and run them on various hardware platforms.

Running a native PyTorch (.pt) model directly on edge devices is slow and uses too much memory. We chose ONNX for deployment due to three main reasons:

- No PyTorch dependency: The system does not need the heavy PyTorch library. It only uses onnxruntime, which is a very lightweight and fast inference engine.
- Hardware acceleration: ONNX Runtime supports hardware acceleration on various edge platforms (such as CPUs, GPUs, and Intel OpenVINO) to speed up processing.

- Resource efficiency: The exported ONNX graph contains only the inference computation graph without training-related operations, reducing runtime overhead and memory usage.

3.5.2. Quantization Evaluation

To find the best balance between processing speed and detection accuracy on edge devices, we tested the model under different quantization conditions. During the experiment, three deployment formats were compared:

1. *Native PyTorch FP32:* The original trained model. It provides the highest baseline accuracy but is too heavy and slow for real-time edge processing.
2. *ONNX FP32:* ONNX FP32 generally provides lower inference latency than the native PyTorch model because it is executed by the optimized ONNX Runtime inference engine.
3. *ONNX INT8 (using QAT):* A highly compressed 8-bit model using Quantization-Aware Training. It is extremely fast and small, but often comes with a drop in detection accuracy.

Experimental results indicated that while the ONNX INT8 format achieved the highest speed, the loss in detection accuracy (mAP drop) was not ideal for strict safety requirements. On the other hand, the ONNX FP32 format provided the most favorable balance. It successfully maintained the high accuracy of the original PyTorch model while delivering excellent inference speed on edge hardware. Therefore, ONNX FP32 was officially selected as the final deployment format for our pipeline.

3.5.3. Temporal Smoothing

In real-time computer vision, single-frame predictions are often unstable. A model might incorrectly detect a violation for a split second due to lighting changes, motion blur, or camera noise. If the system triggers an alarm for every single frame, it will create too many false alarms (false positives). Temporal smoothing solves this problem by analyzing a series of frames over time, rather than trusting a single frame.

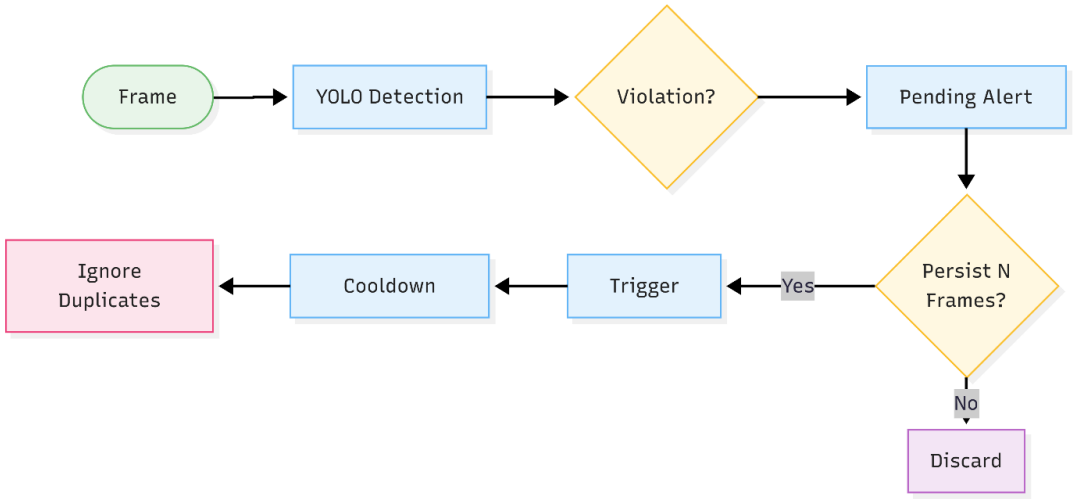


Figure 3.10 - Temporal Smoothing Process

Our system uses a zone-based alert state machine combined with consecutive-frame confirmation. The flow works in three specific steps:

- *Buffering*: When a safety violation is detected, the system does not send an alert immediately. Instead, the alert state is temporarily held in a buffer.
- *Consecutive Confirmation:* The system checks if this exact violation continues to happen in the following frames. An event is only confirmed if the violation persists across multiple consecutive frames.
- *Cooldown Period:* Once an alert is confirmed and reported, a cooldown timer (e.g., 5 seconds) is activated. During this time, repeated alerts for the same person in the same zone are ignored.

This architectural design is strictly implemented for two critical reasons:

- *Improving reliability:* The consecutive confirmation mechanism strictly filters out random, single-frame errors, ensuring the system only outputs highly confident WARNING and CRITICAL events.
- *Preventing system overload:* The cooldown mechanism prevents the edge device from spamming the cloud server with duplicate logs, saving network bandwidth and server storage.

3.6. Edge Deployment

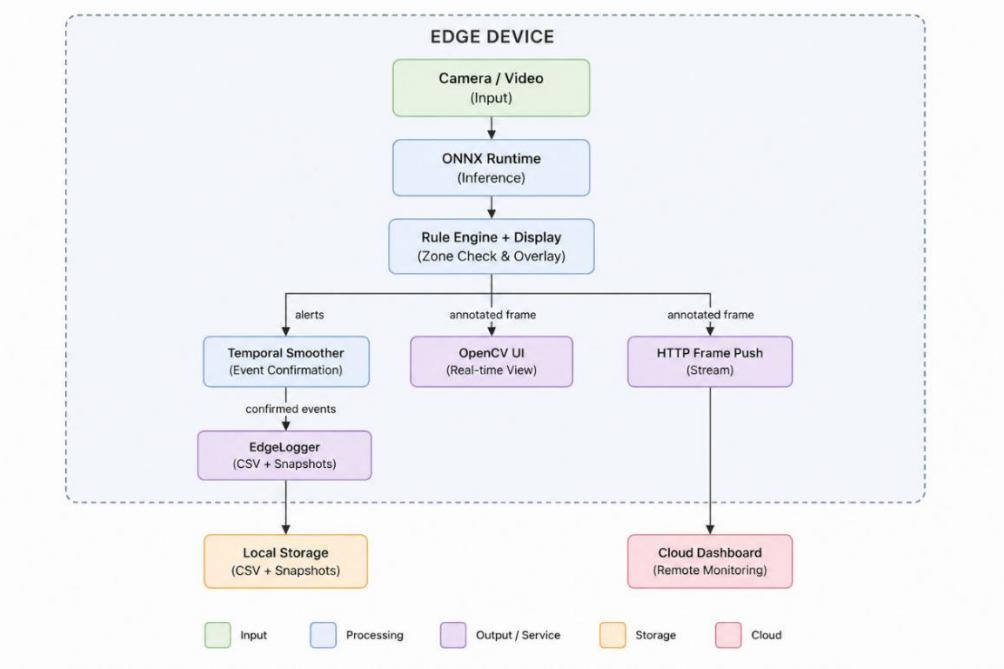


Figure 3.11 - Edge Deployment Architecture

The optimized ONNX model is deployed entirely on the edge device. This section describes the runtime environment, the bandwidth saving strategy, and the output channels of the system.

3.6.1. Runtime Stack

On the edge device, the system uses ONNX Runtime as the core inference engine. When a compatible NVIDIA GPU is available, CUDAExecutionProvider is automatically selected for hardware acceleration. Otherwise, the system falls back to CPUExecutionProvider, ensuring the pipeline can run on any machine.

The pipeline supports two types of input. It can connect to a live camera stream by specifying the device index (e.g., --source 0), or it can process a pre-recorded video file (e.g., --source demo_video3.mp4). The input is opened using OpenCV's VideoCapture, which automatically handles both cases.

Not every frame goes through the neural network. The system can control how often inference is executed. For example, if set to 3, the model runs on every 3rd frame and reuses cached detections for the frames in between. This dramatically reduces compute load on the edge device.

3.6.2. Bandwidth Optimization

Continuously streaming raw video (e.g., 1080p at 2 Mbps) from camera to cloud consumes massive bandwidth. Over 8 working hours, a single camera would generate approximately 7,200 MB of network traffic. This is not scalable for real-world construction sites with multiple cameras.

Instead of transmitting raw video, our system adopts an edge-first architecture. All heavy processing (inference, zone checking, temporal smoothing) happens locally on the edge device. The device only sends data to the server when a confirmed violation event occurs.

Each confirmed event contains only two items:

- *Metadata:* A small JSON/CSV log entry (~500 bytes) recording the timestamp, camera ID, frame ID, zone ID, alert level, violation type, confidence score, and bounding box coordinates.
- *A snapshot image:* A single compressed JPEG image (~300 Kb) of the violation frame, not the entire video stream.

3.6.3. Output Interface

The deployment framework provides three output channels, each serving a different purpose:

Local Event Logging (EdgeLogger)

All confirmed events are written to a CSV file stored directly on the edge device's disk. Each row records the full event details (timestamp, camera, zone, alert level, violations, bbox). A snapshot image is also saved to a local directory. This ensures no data is lost even if the network connection is temporarily unavailable.

Real-Time On-Site Visualization

For on-site security staff, the system renders a real-time video window using OpenCV. The display includes:

- Color-coded bounding boxes: green for NORMAL, orange for WARNING, red for CRITICAL.
- Safety zone polygons drawn as overlays on the video feed.

- PPE violation count labels above each detected person.

This allows security personnel to monitor the situation visually without any network dependency.

Figure 3.12 - Snapshot of a violated scene with drawn bounding boxes

Remote dashboard

For remote monitoring, the system provides two mechanisms to send data to a cloud-based dashboard:

- *Live frame file:* A compressed JPEG frame is periodically written to a shared path, which the dashboard can poll.
- *HTTP push:* Alternatively, the edge device can POST compressed frames directly to a REST endpoint. A built-in retry mechanism stops pushing after 3 consecutive failures to avoid wasting resources.

CHAPTER 4. EXPERIMENTS AND RESULTS

4.1. Model Ablation Study:

An ablation study was conducted to identify the optimal YOLO26n variant for edge deployment. The evaluation prioritized a balance between overall accuracy, specific violation detection capability, and inference speed.

4.1.1. Setup:

- Modal Labs: NVIDIA L4 GPU (22,563 MiB VRAM)
- Ultralytics version 8.4.45
- PyTorch 2.11.0+cu130.
- Performance was measured against a validation set of 2,663 images.

Rank	Experiment	mAP @0.5	Violation mAP@0.5	Infer (ms/img)	Deploy recommendation
1	**baseline_yolo26n**	0.677	0.580	**2.93**	Edge deploy — fastest top-tier violation model
2	baseline_violation _oversample	**0.690**	**0.586**	3.18	Paper/report — best absolute mAP, slightly slower
3	baseline_violation _aug	0.670	0.557	2.95	Not recommended — augmentation underperforms
4	ghost_violation_ oversample	0.629	0.556	3.21	Ablation only
5	cbam_violation_ oversample	0.642	0.553	3.15	Ablation only
6	cbam_existing_ repro	0.618	0.525	3.15	Not recommended
7	ghost_existing_ repro	0.613	0.518	3.11	Not recommended

Table 4.1 - Summary of model ablation study

Based on the validation results, the baseline_yolo26n model was selected as the optimal choice for edge deployment. It recorded an overall mean Average Precision (mAP@0.5)

of 0.677 and a violation-specific mAP@0.5 of 0.580. Crucially, it achieved the fastest inference time among the top-tier models at 2.93 ms per image.

While the baseline_violation_oversample variant achieved the highest absolute mAP@0.5 (0.690) and violation mAP (0.586), it resulted in a slightly slower inference time (3.18 ms per image). Data augmentation experiments (baseline_violation_aug) underperformed, reducing the overall mAP to 0.670. Therefore, the standard baseline was prioritized to maximize deployment speed.

4.1.2. Key per-class mAP@0.5 (baseline_yolo26n)

A per-class evaluation of the baseline_yolo26n model demonstrates high accuracy across core safety equipment categories. The model effectively detects both the presence and absence of critical PPE:

Class	mAP@0.5	Class	mAP@0.5
helmet	0.871	no_helmet	0.800
goggles	0.916	no_goggle	0.817
gloves	0.867	no_gloves	0.744
vest	0.671	no_boots	0.002*
boots	0.682	Person	0.694

Table 4.2 - Key per-class mAP@0.5 using baseline_yolo26n model

A significant limitation was observed in the no_boots class, which yielded an mAP@0.5 of 0.002. An analysis of the data distribution confirmed this is due to severe under-representation; the validation set contained only 2 images with 4 total instances of missing boots. This specific class requires additional targeted data collection.

4.1.3. Architectural Modifications (GhostConv)

Variant	Params	GFLOPs	Val mAP@0.5	Param reduction
Baseline	~2.35 M	~5.0	**0.786**	—
CBAM	~2.31 M	~5.2	0.656	−1.7%
GhostConv	**2.05 M**	**4.4**	0.673	**−12.7%**

Table 4.3 - Comparison of Architectural Modifications

To explore further computational optimization, the baseline architecture was modified using Convolutional Block Attention Module (CBAM) and layers.

GhostConv reduces model size by 12.7% and GFLOPs by 12% but does not improve accuracy, so the baseline remains the deployment choice.

4.2. Edge Quantization Benchmark:

The model was exported to the ONNX format to evaluate the trade-offs between inference latency, model size, and accuracy retention across different precision types (FP32, FP16, and INT8).

Setup

- Kaggle - NVIDIA GPU
- ONNX Runtime 1.26.0
- CUDAExecutionProvider

Variant	Best batch	Latency (ms/img)	FPS	Model size	mAP@0.5	Violation mAP@0.5
FP32 fixed B=1	1	8.99	111	9.4 MB	0.747	0.660
FP32 dynamic	**8**	**7.61**	**131**	10.3 MB	0.749	**0.671**
FP16 dynamic	4	9.05	110	**5.7 MB**	0.750	0.671
INT8 dynamic	1	268.7	3.7	3.7 MB	0.757	0.711

Table 4.4 - Edge Quantization Result

FP32 Dynamic (Batch = 8): Selected as the primary deployment format. Achieving 131 FPS, it stably processes four 30 FPS camera streams concurrently on a single GPU.

FP16 Dynamic: Recommended for memory-constrained devices (Jetson Nano / Orin NX 8 GB). It reduces the model size by 44% (to 5.7 MB) while maintaining identical accuracy.

INT8 Dynamic: Excluded from deployment. Without TensorRT compilation, it is counter-productive on CUDA, running 30× slower than the FP32 reference.

4.3. Temporal Smoothing for Alert Stability

To mitigate transient frame-level false alarms, a temporal smoothing mechanism was integrated into the pipeline. This logic requires a specific zone violation to persist for a consecutive number of frames (N) before raising a confirmed event.

4.3.1. Setup

- Kaggle NVIDIA GPU environment
- Ultralytics YOLO architecture combined
- rule_engine zone integration
- Tested on two sample videos: demo_video3.mp4 (153 frames) and demo_video4.mp4 (144 frames).

4.3.2. Frame-Level vs. Event-Level Alerts

Video:	**demo_video3**	**demo_video4**
Total frames	153	144
Frame-level alerts	17 W / 0 C	96 W / 28 C
N= 3 events	2 W	8 total
N= 3 reduction	88.2%	93.5%
N= 5 events	1 W	6 total
N= 5 reduction	**94.1%**	**95.2%**
N= 10 events	1 W	5 total
N= 10 reduction	94.1%	**96.0%**

Table 4.5 - Temporal Smoothing Experiment Result

Operating without temporal smoothing produced a high volume of unstable alerts. For demo_video3, the system generated 17 warning alerts and 0 critical alerts. For demo_video4, it generated 96 warning and 28 critical alerts.

4.3.3. Recommended setting

N_frames	Avg FP reduction	Alert latency @ 30 FPS
3	90.9%	~100 ms
5	**94.6%**	**~167 ms**
10	95.0%	~333 ms

Table 4.6 - Performance Evaluation of Temporal Smoothing Thresholds

Deployment Recommendation The **N = 5** configuration is selected for final deployment. It delivers a 94.6% reduction in false alarms while adding only ~167 ms of latency, comfortably satisfying the system's <200 ms real-time alert target. Increasing the threshold to N = 10 yields only a marginal 0.4% gain in reduction while doubling the delay, which is impractical for immediate safety interventions.

4.4. Bandwidth Optimization Analysis

To overcome the network constraints of continuous cloud streaming, the pipeline's edge-processed data transmission was evaluated against raw video streaming.

Scenario	Data / hour	vs. Raw video
A - Raw demo video stream	1943 MB	baseline
A ref - 1080p @ 2 Mbps stream	858 MB	−55.8%
B - Edge logs + violation snapshots	**1.35 MB**	**−99.93%**
C - Logs + blurred critical frames	**0.39 MB**	**−99.98%**

Table 4.7 - Bandwidth Optimization Analysis Result

The edge-first approach (Scenario B) achieves a 99.93% reduction in upstream bandwidth. This minimal data footprint validates the system's operational viability over standard 4G/LTE mobile networks commonly available at construction sites.

4.5. Deployment Recommendation Summary

Based on the evaluations of model architecture, hardware quantization, alert stability, and network constraints, the following configurations are established as the optimal parameters for the deployment pipeline:

Dimension	Recommended choice	Key metric
Model variant	baseline_yolo26n	val mAP@0.5 = 0.786 2.93 ms/img
Export format (GPU edge)	FP32 dynamic ONNX, B=8	131 FPS
Export format (RAM-limited)	FP16 dynamic ONNX	5.7 MB, same accuracy
Alert smoothing	N = 5 consecutive frames	94.6% FP reduction 167 ms latency
Bandwidth mode	Scenario B (logs + snapshots)	1.35 MB/hour (−99.93%)

Table 4.8 - Deployment Recommendation Settings

4.6. Dashboard

A web-based monitoring dashboard was developed as the primary user interface for the system. The dashboard is built with Flask (Python) on the backend and vanilla JavaScript with Chart.js on the frontend. It is lightweight enough to run directly on the edge device alongside the inference pipeline.

4.6.1. Live Monitor

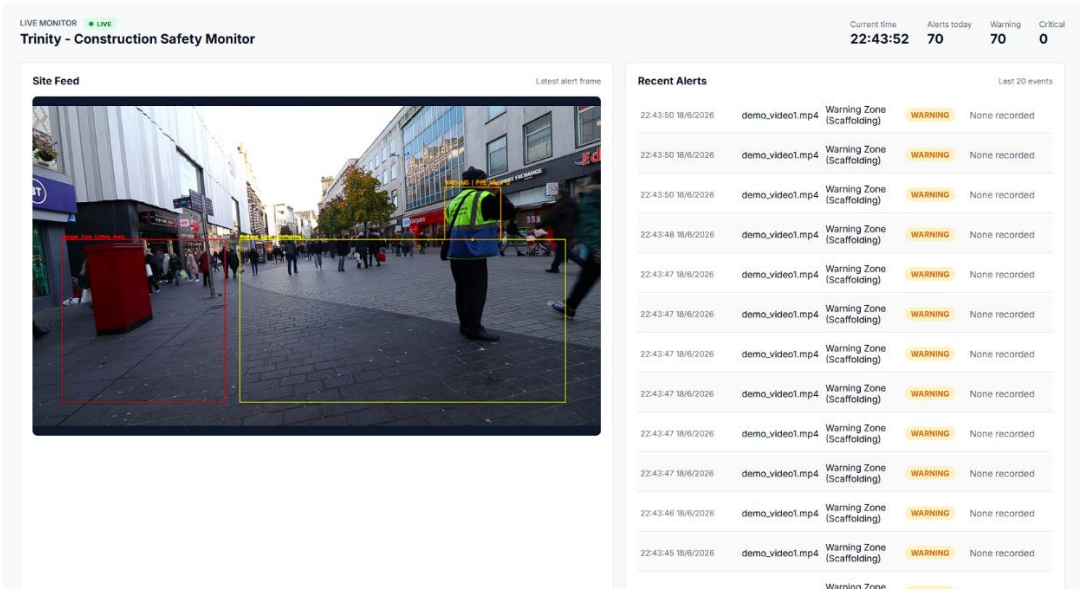


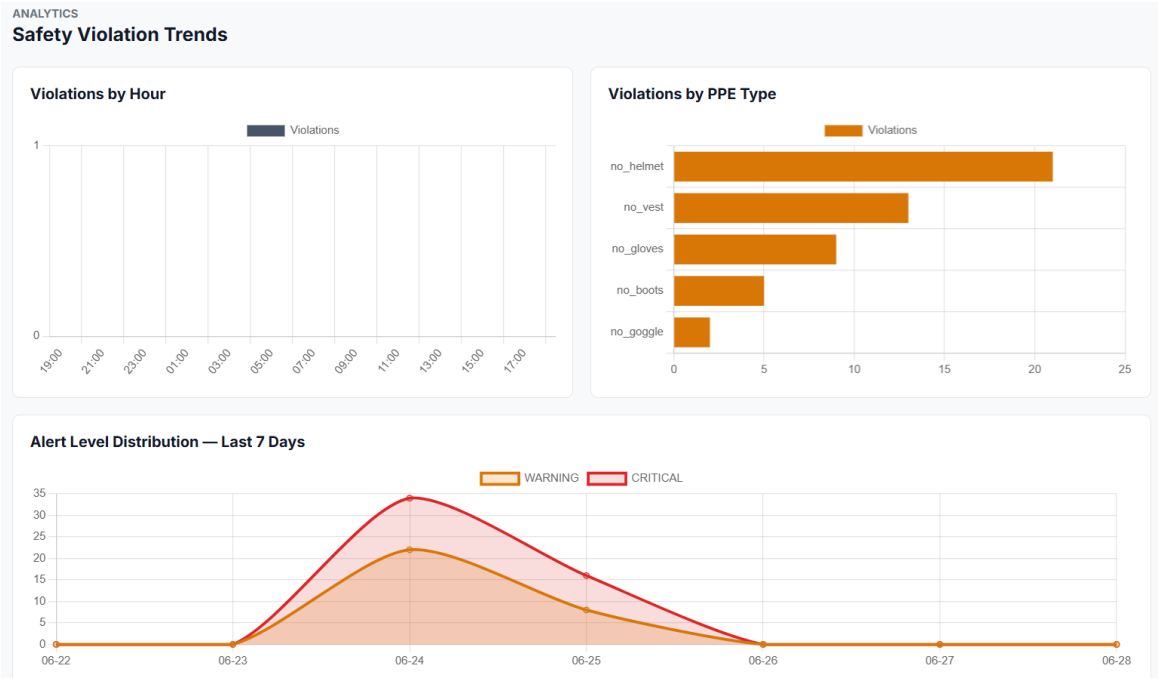
Figure 4.1 - Live monitor dashboard UI

The main page displays a real-time camera feed and the 20 most recent alert events. When the edge pipeline is actively running, the feed shows a live MJPEG video stream.

When the pipeline is idle or completed, the feed falls back to the latest available snapshot. Summary metrics at the top of the page show the total number of alerts, warnings, and critical events for the current day. These counters auto-refresh every 3 seconds. Clicking on any alert row opens a modal window displaying the corresponding violation snapshot image.

4.6.2. Analytics

The analytics page provides four visual components for trend analysis: a bar chart showing violation counts grouped by hour over the last 24 hours, a horizontal bar chart breaking down violations by PPE type (no_helmet, no_vest, no_gloves, no_boots, no_goggle), a line chart tracking WARNING and CRITICAL alert volumes over the past 7 days, and a zone-violation heatmap table that cross-references each safety zone against each PPE violation category. All charts are rendered client-side using Chart.js and update automatically when new event data is available.



Zone Violation Heatmap

ZONE	NO_HELMET	NO_VEST	NO_GLOVES	NO_BOOTS	NO_GOGGLE
DANGER ZONE (LIFTING AREA)	0	0	0	0	0
GROUND LEVEL GENERAL	7	6	6	4	0
MACHINERY AREA	6	1	0	0	0
RESTRICTED ZONE A	6	2	0	0	0
WARNING ZONE (SCAFFOLDING)	0	0	0	0	0
WORKING AT HEIGHT	2	4	3	1	2

Figure 4.2 - Analytics UI

4.6.3. Event log

The event log page presents a searchable, sortable, and paginated table of all recorded violations. Users can filter events by date range, camera ID, zone name, alert level, and PPE type. Columns can be sorted by timestamp, alert level, or confidence score. A one-click CSV export function is provided for offline reporting and compliance documentation.

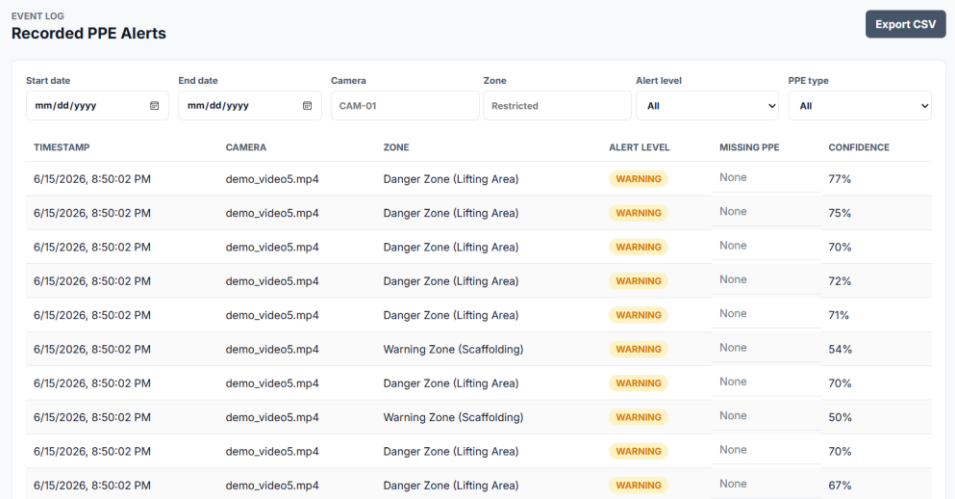


Figure 4.3 - Event log UI

4.6.4. System status

The status page displays the health of connected edge nodes, including camera ID, location, online/offline status, current FPS, and last heartbeat. A model information panel shows the deployed model name (baseline_yolo26n), format (ONNX FP32), inference speed (131 FPS), and temporal smoothing setting (N=5). A summary banner reports three key operational metrics: 99.93% bandwidth saving, 94.6% false alarm reduction, and <167 ms alert latency

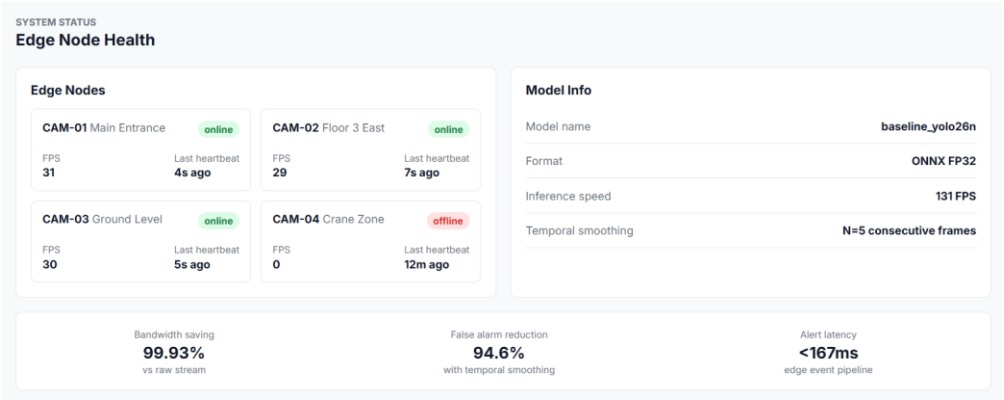


Figure 4.4 - System status UI

CHAPTER 5. DISCCUSION

5.1. Advantages

5.1.1. Context-aware spatial alerting

Standard PPE detection systems operate on binary logic: equipment is either present or missing, regardless of where the worker is standing. This often produces irrelevant alerts, for example when a worker removes their helmet in a designated rest area. The proposed system addresses this limitation by introducing a zone-based rule engine. Using OpenCV's pointPolygonTest, the pipeline maps each detected worker to predefined safety polygons (Danger, Warning, or Outside zones) and combines the spatial context with the PPE detection result to determine alert severity. As a result, the system can distinguish between a genuine violation in a restricted area and a harmless action in a safe zone, reducing unnecessary notifications for safety managers.

5.1.2. Efficient edge inference

The system is designed with the computational constraints of edge hardware in mind. Due to hardware availability constraints during this phase, the software pipeline was benchmarked using a cloud GPU proxy. Through ablation testing, the lightweight baseline_yolo26n architecture was selected over attention-based variants. When exported to ONNX FP32 with dynamic batching (Batch=8), the pipeline reached a throughput of 131 FPS. While absolute performance on physical edge devices will vary, this baseline validates the pipeline's computational efficiency and readies the system for future deployment on physical multi-camera edge nodes**.**

5.1.3. False alarm suppression via temporal smoothing

Single-frame detection errors are common in real-world video due to motion blur, lighting changes, and partial occlusions. If every such error triggers an alert, the system becomes unreliable. To address this, a 5-frame Temporal Smoothing algorithm was implemented. Experiments on test footage showed that requiring a violation to persist across 5 consecutive frames reduced false positive alerts by 94.6%. The additional latency introduced by this mechanism was approximately 167 ms, which remains well within the sub-200 ms threshold generally expected for real-time safety intervention systems.

5.1.4. Bandwidth efficiency through edge-first architecture

A key operational advantage of the system is its edge-first data transmission strategy. Continuous 1080p video streaming consumes approximately 1,943 MB per hour per camera. The proposed architecture avoids this by processing all video locally on the edge device and transmitting only confirmed violation events. Each event payload consists of a structured JSON log and a single compressed JPEG snapshot. Bandwidth analysis showed that this approach consumes only 1.35 MB per hour, a 99.93% reduction compared to continuous streaming. This low data footprint makes the system practical for deployment over standard 4G/LTE mobile networks, which are common on remote construction sites where wired infrastructure is unavailable.

5.2. Limitations

5.2.1. Class imbalance and targeted detection failure

Although the model performs well on core PPE classes, it exhibits a critical failure in detecting the no_boots class, which yielded a validation mAP@0.5 of only 0.002. Data distribution analysis confirmed that this is caused by severe under-representation; the validation set contained only 2 images with 4 total instances of missing boots. This limitation highlights a vulnerability in the current dataset, indicating that heavily imbalanced or rare violation states cannot be effectively detected without targeted, field-specific data collection.

5.2.2. Hardware acceleration dependency and quantization limits

The system's optimal inference throughput (131 FPS) was benchmarked on a cloud-based NVIDIA GPU proxy environment. Because physical testing on the target edge hardware (e.g., NVIDIA Jetson Xavier NX) was not conducted during this phase, actual inference speeds in field deployments will inevitably be lower than this benchmark. Maintaining real-time performance on-site necessitates edge devices equipped with dedicated GPU accelerators, which increases the deployment cost per camera node compared to basic CPU-based IoT devices. Furthermore, experiments with INT8 dynamic quantization failed to improve speed under the current ONNX Runtime setup. Without hardware-specific TensorRT compilation, the INT8 format ran at 3.7 FPS

(approximately 30× slower than the FP32 reference), meaning the pipeline cannot currently fully exploit 8-bit compression for memory-constrained devices.

5.2.3. Static 2D spatial configuration

The context-aware rule engine relies on static 2D polygons defined in a local JSON configuration file. While effective for fixed camera feeds, this approach lacks spatial adaptability. If a camera is accidentally bumped, repositioned, or if the physical hazard zone changes as construction progresses, the predefined polygons will no longer align with the actual physical environment. Updating the zones currently requires manual recalibration by an operator, as the system does not yet integrate dynamically with 3D Building Information Modeling (BIM) data.

5.2.4. Susceptibility to environmental conditions

As a computer vision pipeline relying entirely on standard RGB camera feeds, the system's detection accuracy is inherently tied to visual clarity. While the model handles typical daytime industrial lighting, its performance is expected to degrade in extreme environmental conditions common on active construction sites, such as heavy rain, thick dust, or nighttime operations without adequate artificial illumination. The current architecture relies solely on optical data and lacks multi-modal sensor fusion (such as thermal or LiDAR sensors) to compensate for these visual limitations.

5.3. Contributions

This research makes four contributions to the field of automated construction safety monitoring, addressing several limitations identified in existing literature.

5.3.1. Comprehensive dataset synthesis

A key contribution of this project is addressing the class imbalance found in publicly available PPE datasets. By remapping and merging three distinct sources (Ultralytics, PPE Combined, and Construction Site Safety), the project produced a training set of over 30,000 images. This integration expanded the representation of missing-PPE violation classes (such as no_helmet, no_goggle, and no_gloves), aligning with industrial safety requirements without including unrelated object categories.

5.3.2. Context-aware safety framework

This research moves beyond binary object detection by developing a context-aware safety framework. By combining the YOLO26n detections with a spatial rule engine and a 5-frame Temporal Smoothing algorithm, the system evaluates the physical location of workers before assigning an alert severity. In experiments, this design reduced transient false positive alarms by 94.6%, ensuring that alerts correspond to genuine spatial risks rather than momentary detection errors.

5.3.3. Empirical validation of edge inference performance

The project provides empirical data on deploying deep learning models in edge computing environments. Rather than relying only on theoretical accuracy metrics, the study conducted an ablation and quantization benchmark comparing multiple model variants and precision formats. Results showed that exporting the baseline_yolo26n model to ONNX FP32 with dynamic batching offered a favorable speed-accuracy trade-off, achieving a throughput of 131 FPS and a latency of 2.93 ms per image on a simulated edge GPU.

5.3.4. Bandwidth-efficient edge-first architecture

The research delivers a complete edge-first deployment pipeline that addresses the network constraints of cloud-centric monitoring. By processing all video locally and transmitting only structured metadata logs and compressed violation snapshots, the system achieves a 99.93% reduction in upstream bandwidth consumption (1.35 MB/hour versus 1,943 MB/hour for continuous streaming). This result provides a practical reference for deploying multi-camera AI monitoring systems over standard 4G/LTE mobile networks.

5.4. Future work

To address the current limitations and extend the system's applicability, future development should focus on the following directions.

5.4.1. Data collection for underrepresented classes

The current model fails to detect the no_boots class (mAP@0.5 of 0.002) due to having only 2 validation images with 4 annotated instances. Future work should prioritize field-specific data collection campaigns, capturing real-world footage of workers with and

without safety boots across diverse angles and lighting conditions. Additionally, applying synthetic data augmentation techniques could help bootstrap the training set for rare violation classes without requiring large-scale manual annotation.

5.4.2. INT8 quantization with hardware-specific compilation

The current INT8 dynamic quantization path produced unexpectedly slow inference (3.7 FPS) under the generic ONNX Runtime setup. Future work should explore hardware-specific compilation pipelines such as NVIDIA TensorRT, which performs layer fusion and kernel auto-tuning for the target GPU architecture. This would allow the system to fully exploit 8-bit compression, further reducing memory usage and potentially enabling deployment on lower-cost edge devices without dedicated GPU accelerators.

5.4.3. Dynamic zone generation via bim integration

The current zone configuration relies on static 2D polygons stored in a local JSON file, which requires manual recalibration whenever the camera or the physical environment changes. Integrating the system with Building Information Modeling (BIM) data would allow safety zones to be automatically generated and updated as the construction site evolves. This would eliminate the dependency on manual polygon drawing and enable the system to adapt to dynamic site layouts without operator intervention.

5.4.4. Multi-modal sensor fusion for adverse conditions

Since the pipeline relies entirely on standard RGB cameras, its accuracy is expected to degrade under poor visibility conditions such as heavy rain, thick dust, or nighttime operations. Future iterations should investigate the integration of complementary sensor modalities, including thermal imaging for low-light detection and LiDAR-based depth sensing for occlusion handling. Fusing these inputs with the existing RGB pipeline would improve detection robustness across a wider range of environmental conditions.

5.4.5. Multi-camera person re-identification

The current system processes each camera feed independently, meaning that if a worker in violation moves between two overlapping camera views, the same violation may be logged twice. Implementing a Person Re-Identification (ReID) module would allow the system to track individuals across multiple camera feeds, preventing duplicate alerts and enabling site-wide worker tracking from a single unified dashboard.

CONCLUSION

This research presented an end-to-end edge computing pipeline for real-time Personal Protective Equipment (PPE) violation detection on construction sites. The system was designed to address the practical limitations of traditional cloud-based monitoring, including high network bandwidth consumption, processing latency, and worker privacy concerns. The pipeline integrates a lightweight YOLO26n object detection model with a context-aware spatial rule engine, a temporal smoothing algorithm, and an edge-first data transmission architecture.

The experimental results validated the effectiveness of the proposed approach across multiple dimensions. The selected baseline_yolo26n model, exported to ONNX FP32, achieved an inference throughput of 131 FPS with a latency of 2.93 ms per image, confirming its suitability for real-time multi-camera edge deployments. The 5-frame Temporal Smoothing mechanism reduced false positive alerts by 94.6% while introducing only 167 ms of additional latency. At the network level, the edge-first architecture reduced upstream bandwidth consumption by 99.93% (from 1,943 MB/hour to 1.35 MB/hour per camera), making the system viable for deployment over standard 4G/LTE mobile networks on remote construction sites.

Despite these results, the study identified several limitations that warrant further investigation. The model exhibited poor detection performance on the no_boots class (mAP@0.5 of 0.002) due to insufficient training samples, and the INT8 quantization path did not yield the expected speed gains under the current ONNX Runtime configuration. Additionally, the static 2D zone configuration requires manual recalibration when cameras are repositioned, and the RGB-only input limits reliability under adverse weather or low-light conditions.

Future work should focus on targeted data collection for underrepresented classes, hardware-specific INT8 compilation via TensorRT, dynamic zone generation through BIM integration, and multi-modal sensor fusion to extend the system's operational range. These improvements would bring the pipeline closer to a production-grade safety monitoring solution suitable for large-scale, multi-site industrial deployment.

REFERENCES

[1] M. Hien, “Tình hình tai nạn lao động năm 2025: Giảm về số vụ nhưng vẫn tiềm ẩn nhiều rủi ro.,” Tạp chí Tổ chức nhà nước và Lao động - Bộ Nội vụ, Mar. 04, 2026. [Online]. Available: https://tcnnld.vn/news/detail/71676/Tinh-hinh-tai-nan-lao-dong-nam-2025-Giam-ve-so-vu-nhung-van-tiem-an-nhieu-rui-ro.html

[2] H. Xinyu and H. Jimmie, “Analysis of construction worker fall accidents,” Jun. 2003.

[3] S. JoonOh, H. SangUk, L. SangHyun, and K. Hyoungkwan, “Computer vision techniques for construction safety and health monitoring,” vol. 29, pp. 239–251, 2015.

[4] N. D. Nath, A. H. Behzadan, and S. G. Paal, “Deep learning for site safety: Real-time detection of personal protective equipment,” *Autom. Constr.*, vol. 112, p. 103085, Apr. 2020, doi: 10.1016/j.autcon.2020.103085.

[5] W. Fang, L. Ding, H. Luo, and P. E. D. Love, “Falls from heights: A computer vision-based approach for safety harness detection,” *Autom. Constr.*, vol. 91, pp. 53–61, Jul. 2018, doi: 10.1016/j.autcon.2018.02.018.

[6] S. Woo, J. Park, J.-Y. Lee, and I. S. Kweon, “CBAM: Convolutional Block Attention Module,” 2018, *arXiv*. doi: 10.48550/ARXIV.1807.06521.

[7] Q. Hou, D. Zhou, and J. Feng, “Coordinate Attention for Efficient Mobile Network Design,” 2021, *arXiv*. doi: 10.48550/ARXIV.2103.02907.

[8] K. Han, Y. Wang, Q. Tian, J. Guo, C. Xu, and C. Xu, “GhostNet: More Features from Cheap Operations,” Mar. 13, 2020, *arXiv*: arXiv:1911.11907. doi: 10.48550/arXiv.1911.11907.

[9] NVIDIA Corporation, “Jetson Xavier NX Developer Kit.” 2024. [Online]. Available: https://developer.nvidia.com/embedded/learn/get-started-jetson-xavier-nx-devkit

[10] Z. Wu, X. Lei, and M. Kumar, “Advancing construction safety: YOLOv8-CGS helmet detection model,” *PLOS One*, vol. 20, no. 5, p. e0321713, May 2025, doi: 10.1371/journal.pone.0321713.

[11] C. Song and Y. Li, “A YOLOv8 algorithm for safety helmet wearing detection in complex environment,” *Sci. Rep.*, vol. 15, no. 1, p. 24236, Jul. 2025, doi: 10.1038/s41598-025-08828-z.

[12] B. Tong, G. Li, X. Bu, Y. Wang, and X. Yu, “A deep learning-based algorithm for the detection of personal protective equipment,” *PLOS One*, vol. 20, no. 5, p. e0322115, May 2025, doi: 10.1371/journal.pone.0322115.

[13] X. Xiao, C. Chen, M. Skitmore, H. Li, and Y. Deng, “Exploring Edge Computing for Sustainable CV-Based Worker Detection in Construction Site Monitoring: Performance and Feasibility Analysis,” *Buildings*, vol. 14, no. 8, p. 2299, Jul. 2024, doi: 10.3390/buildings14082299.

[14] M. Gugssa, L. Li, L. Pu, A. Gurbuz, Y. Luo, and J. Wang, “Enhancing the Time Efficiency of Personal Protective Equipment (PPE) Detection in Real Implementations Using Edge Computing,” in *Computing in Civil Engineering 2023*, Corvallis, Oregon: American Society of Civil Engineers, Jan. 2024, pp. 532–540. doi: 10.1061/9780784485248.064.

[15] C. Chen, H. Gu, S. Lian, Y. Zhao, and B. Xiao, “Investigation of Edge Computing in Computer Vision-Based Construction Resource Detection,” *Buildings*, vol. 12, no. 12, p. 2167, Dec. 2022, doi: 10.3390/buildings12122167.

[16] G. Gallo, F. D. Rienzo, F. Garzelli, P. Ducange, and C. Vallati, “A Smart System for Personal Protective Equipment Detection in Industrial Environments Based on Deep Learning at the Edge,” *IEEE Access*, vol. 10, pp. 110862–110878, 2022, doi: 10.1109/ACCESS.2022.3215148.

[17] D. G. Lema, R. Usamentiaga, and D. F. García, “Low-cost system for real-time verification of personal protective equipment in industrial facilities using edge computing devices,” *J. Real-Time Image Process.*, vol. 20, no. 6, p. 111, Dec. 2023, doi: 10.1007/s11554-023-01368-7.

[18] S. Adam, “Privacy-Preserving AI Techniques for Edge Devices,” Jun. 18, 2024. [Online]. Available: https://dialzara.com/blog/privacy-preserving-ai-techniques-for-edge-devices

[19] “Privacy-preserving AI at the edge,” XenonStack, Nov. 2024. [Online]. Available: https://www.xenonstack.com/blog/privacy-preserving-ai-edge

[20] Fluendo Blog, “Privacy-preserving AI video surveillance with edge intelligence,” Jun. 2025. [Online]. Available: https://fluendo.com/blog/privacy-preserving-ai-video-surveillance-with-edge-intelligence/

[21] J. E. Rivadeneira, G. A. Borges, A. Rodrigues, F. Boavida, and J. Sá Silva, “A unified privacy preserving model with AI at the edge for Human-in-the-Loop Cyber-Physical Systems,” *Internet Things*, vol. 25, p. 101034, Apr. 2024, doi: 10.1016/j.iot.2023.101034.

[22] N. Nipun D., B. Amir H., and P. Stephanie G., “Deep learning for site safety: Real-time detection of personal protective equipment”.

[23] Roboflow 100, “Construction Safety Dataset.” Roboflow, May 2023. Accessed: Jun. 30, 2026. [Open Source Dataset]. Available: https://universe.roboflow.com/roboflow-100/construction-safety-gsnvb

[24] R. U. Projects, “Personal Protective Equipment - Combined Model Dataset,” *Roboflow Universe*. Roboflow, Jun. 2025. [Online]. Available: https://universe.roboflow.com/roboflow-universe-projects/personal-protective-equipment-combined-model

[25] R. U. Projects, “Construction Site Safety Dataset,” *Roboflow Universe*. Roboflow, Jan. 2026. [Online]. Available: https://universe.roboflow.com/roboflow-universe-projects/construction-site-safety

[26] M. Dalvi, N. Singh, S. Bhingarde, and K. Chalke, “Construction-PPE: Personal Protective Equipment Detection Dataset.” Ultralytics, Jan. 2025. [Online]. Available: https://docs.ultralytics.com/datasets/detect/construction-ppe/

[27] “Tips for Best YOLOv5 Training Results,” Ultralytics Docs. Accessed: Jun. 30, 2026. [Online]. Available: https://docs.ultralytics.com/yolov5/tutorials/tips-for-best-training-results

[28] R. Sapkota, R. H. Cheppally, A. Sharda, and M. Karkee, “YOLO26: Key Architectural Enhancements and Performance Benchmarking for Real-Time Object Detection,” Mar. 16, 2026, *arXiv*: arXiv:2509.25164. doi: 10.48550/arXiv.2509.25164.